

Python + SQL Fundamentals (SQLite3 Complete Core Guide)

This guide teaches the core and fundamentals of SQL in Python using the built-in sqlite3 library. It covers database creation, tables, CRUD operations, filtering, sorting, aggregation, relationships, joins, transactions, exception handling, and best practices.

1. What is SQL and SQLite?

SQL (Structured Query Language) is used to communicate with databases. SQLite is a lightweight relational database that stores everything in a single file. Python provides built-in support for SQLite through the sqlite3 module.

2. Importing Libraries

```
import sqlite3
import os
```

sqlite3 → Allows Python to interact with SQLite databases.
os → Used for file operations such as deleting old database files.

3. Creating a Database Connection

```
conn = sqlite3.connect('students.db')
cursor = conn.cursor()
```

Connection object:

- Represents the database connection.
- Used for commit() and close().

Cursor object:

- Executes SQL commands.
- Fetches records from queries.

4. Creating Tables

```
cursor.execute('''
CREATE TABLE IF NOT EXISTS Students (
id INTEGER PRIMARY KEY,
name VARCHAR NOT NULL,
age INTEGER,
email VARCHAR UNIQUE,
city VARCHAR
)
''')
```

Important SQL Constraints:

- PRIMARY KEY → Unique identifier.
- NOT NULL → Cannot contain NULL values.
- UNIQUE → Prevents duplicate values.

5. SQLite Data Types

SQLite Common Data Types: - INTEGER → Whole numbers - REAL → Decimal numbers - TEXT → Strings - BLOB → Binary data - NULL → Empty values

6. Inserting Data

Single Insert: `cursor.execute("INSERT INTO Students VALUES (1, 'Alice', 20, 'alice@gmail.com', 'NY'))` Multiple Insert: `students = [(1, 'Alice', 20, 'a@gmail.com', 'NY')]`
`cursor.executemany("INSERT INTO Students VALUES (?, ?, ?, ?, ?)", students)` Why use ? placeholders?
- Prevents SQL injection.
- Safer and cleaner.

7. SELECT Queries

Select all records: `SELECT * FROM Students` Select specific columns: `SELECT name, age FROM Students`
Filtering: `SELECT * FROM Students WHERE age = 20`

8. WHERE Conditions

Common Operators:
=, >, <, >=, <=, != Examples: `SELECT * FROM Students WHERE city = 'New York'`
`SELECT * FROM Students WHERE age > 20` Logical Operators:
- AND
- OR
- NOT

9. ORDER BY and LIMIT

Sorting: `SELECT * FROM Students ORDER BY age ASC` Descending: `SELECT * FROM Students ORDER BY age DESC` Limiting Results: `SELECT * FROM Students LIMIT 3`

10. UPDATE Records

`UPDATE Students SET age = 24 WHERE id = 6` Always use WHERE in UPDATE unless you want to update every row.

11. DELETE Records

`DELETE FROM Students WHERE id = 6` Without WHERE, all rows are deleted.

12. Transactions and commit()

`conn.commit()` Changes are not permanently saved until `commit()` is called.

13. Fetch Methods

fetchone() → Returns one row.
fetchall() → Returns all rows.
fetchmany(n) → Returns n rows.

14. Aggregate Functions

COUNT(): SELECT COUNT(*) FROM Students AVG(): SELECT AVG(age) FROM Students MAX/MIN:
SELECT MAX(age), MIN(age) FROM Students SUM(): SELECT SUM(age) FROM Students

15. GROUP BY

SELECT city, COUNT(*) FROM Students GROUP BY city Groups rows having the same value.

16. LIKE Operator

Pattern searching: SELECT * FROM Students WHERE name LIKE 'A%' Wildcards:
% → many characters
_ → one character

17. Joins

INNER JOIN Example: SELECT Students.name, Courses.course_name
FROM Students
INNER JOIN Courses
ON Students.id = Courses.id Types of Joins:
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN (not supported directly in SQLite)
- FULL JOIN (simulated in SQLite)

18. Primary Key vs Foreign Key

Primary Key:
- Uniquely identifies rows. Foreign Key:
- Connects tables together. Example: FOREIGN KEY(student_id) REFERENCES Students(id)

19. Exception Handling

```
try:  
    cursor.execute(query)  
except sqlite3.Error as e:
```

`print(e)` Prevents program crashes and helps debugging.

20. Closing Connection

`conn.close()` Always close the database connection to free resources.

21. Best Practices

- Use parameterized queries (? placeholders).
- Always commit changes.
- Close connections.
- Use IF NOT EXISTS for tables.
- Avoid SELECT * in large applications.
- Use meaningful table names.
- Backup databases regularly.

22. Full Example Workflow

Typical workflow:

1. Import sqlite3
2. Create connection
3. Create cursor
4. Create tables
5. Insert data
6. Query data
7. Update/Delete records
8. Commit changes
9. Close connection

SQL Query Cheat Sheet

Operation	Example
SELECT	SELECT * FROM Students
INSERT	INSERT INTO Students VALUES (...)
UPDATE	UPDATE Students SET age=20 WHERE id=1
DELETE	DELETE FROM Students WHERE id=1
COUNT	SELECT COUNT(*) FROM Students
AVG	SELECT AVG(age) FROM Students
ORDER BY	SELECT * FROM Students ORDER BY age
LIMIT	SELECT * FROM Students LIMIT 5

Congratulations! You now have a strong foundation in SQL using Python and SQLite. Mastering these fundamentals prepares you for advanced databases such as MySQL, PostgreSQL, Oracle, and SQL Server.